

POCKETPILOT — MAIN REPOSITORY SETUP PROMPT

You are setting up the shared repository foundation for **PocketPilot (FutureReceipt)**.

Your job is limited to:

1. Creating the repository and folder structure.
2. Creating project documentation and shared specifications.
3. Creating root configuration files.
4. Creating GitHub ownership and branch-protection support files.
5. Creating empty workstream directories.
6. Creating shared workflow, status, handoff, and integration-request templates.

Do not implement product features.

Do not build:

- Frontend screens
- React components
- Finance calculations
- AI workflows
- API routes
- Database logic
- Authentication
- User accounts
- Production integrations

1. Project Overview

PocketPilot is a pre-spend financial decision companion.

It allows a user to ask questions such as:

Can I afford this ₹59,999 phone on a 12-month EMI?

The product then:

1. Collects the user's financial context.
2. Extracts the proposed purchase details.
3. Runs deterministic cash-flow calculations.
4. Tests the purchase against the user's Money Constitution.

5. Generates multiple possible futures.
6. Shows a Future Receipt explaining the impact.
7. Suggests a safer purchase plan.

PocketPilot is not an expense tracker, traditional budgeting app, banking app, investment product, or payment application.

The MVP is:

- Single-user
 - Demo-first
 - No authentication
 - No accounts
 - No sessions
 - No multi-tenancy
 - No Row-Level Security
 - No payment initiation
 - No direct banking integrations
-

2. Three Workstreams

The project is divided into three isolated workstreams.

Task 1 — Product Experience and Frontend

Owned areas:

```
src/app/**
src/components/**
src/features/**
src/mocks/**
src/styles/**
public/**
tests/e2e/**
docs/workstreams/task-1/**
```

Task 2 — Deterministic Finance and Constitution Engine

Owned areas:

```
src/core/finance/**
tests/finance/**
```

```
tests/fixtures/**
docs/workstreams/task-2/**
```

Task 3 — AI, Contracts, Backend and Integration

Owned areas:

```
src/contracts/**
src/core/ai/**
src/server/**
src/app/api/**
supabase/**
tests/ai/**
tests/integration/**
docs/workstreams/task-3/**
```

Shared files

These require joint review:

```
package.json
pnpm-lock.yaml
tsconfig.json
next.config.ts
vitest.config.ts
playwright.config.ts
.env.example
src/contracts/**
docs/CONTRACT_SPEC.md
docs/CONTRACT_CHANGELOG.md
```

Task 3 implements contracts, but it must not silently introduce breaking changes.

3. Required Repository Structure

Create this structure:

```
pocketpilot/
├─ .github/
│   └─ CODEOWNERS
│   └─ pull_request_template.md
```

```
|   └─ workflows/
|       └─ quality.yml
|       └─ ownership-check.yml
|
|   └─ docs/
|       └─ PRODUCT_OVERVIEW.md
|       └─ MVP_SCOPE.md
|       └─ ARCHITECTURE.md
|       └─ CONTRACT_SPEC.md
|       └─ CONTRACT_CHANGELOG.md
|       └─ DEVELOPMENT_RULES.md
|       └─ INTEGRATION_PLAN.md
|       └─ DEMO_SCENARIO.md
|       └─ AI_SAFETY.md
|       └─ UI_DESIGN_PRINCIPLES.md
|       └─ integration-requests/
|           └─ README.md
|           └─ task-1-requests.md
|           └─ task-2-requests.md
|           └─ task-3-requests.md
|       └─ workstreams/
|           └─ task-1/
|               └─ STATUS.md
|               └─ DECISIONS.md
|               └─ HANDOFF.md
|           └─ task-2/
|               └─ STATUS.md
|               └─ DECISIONS.md
|               └─ HANDOFF.md
|           └─ task-3/
|               └─ STATUS.md
|               └─ DECISIONS.md
|               └─ HANDOFF.md
|
|   └─ scripts/
|       └─ check-ownership.mjs
|       └─ smoke-test.mjs
|       └─ verify-handoff.mjs
|
|   └─ public/
|       └─ demo/
|
|   └─ src/
|       └─ app/
|           └─ api/
|       └─ components/
```

```

├── features/
│   ├── mocks/
│   ├── styles/
│   ├── contracts/
│   ├── core/
│   │   ├── finance/
│   │   └── ai/
│   └── server/
├── supabase/
│   └── migrations/
├── tests/
│   ├── finance/
│   ├── ai/
│   ├── integration/
│   ├── e2e/
│   └── fixtures/
├── .editorconfig
├── .env.example
├── .gitignore
├── .nvmrc
├── AGENTS.md
├── README.md
├── package.json
├── next.config.ts
├── playwright.config.ts
├── tsconfig.json
└── vitest.config.ts

```

Because Git does not preserve empty directories, place `.gitkeep` files inside empty implementation folders.

Do not add product implementation code.

4. Branch Strategy

Document these branches:

```
task-1-product-ui
task-2-finance-engine
task-3-platform-ai
```

No one may push directly to `main`.

Each workstream must:

1. Pull the latest `main`.
2. Work only in its owned paths.
3. Use small commits.
4. Push only to its assigned branch.
5. Open a pull request.
6. Pass tests and ownership checks.
7. Update its continuation documents before stopping.

5. CODEOWNERS

Create `.github/CODEOWNERS` with placeholders:

```
# Replace these placeholders with real GitHub usernames.

# Task 1
/src/app/                @TASK1_OWNER
/src/components/         @TASK1_OWNER
/src/features/           @TASK1_OWNER
/src/mocks/              @TASK1_OWNER
/src/styles/             @TASK1_OWNER
/public/                 @TASK1_OWNER
/tests/e2e/              @TASK1_OWNER
/docs/workstreams/task-1/ @TASK1_OWNER

# Task 2
/src/core/finance/       @TASK2_OWNER
/tests/finance/          @TASK2_OWNER
/tests/fixtures/         @TASK2_OWNER
/docs/workstreams/task-2/ @TASK2_OWNER

# Task 3
/src/contracts/          @TASK3_OWNER
/src/core/ai/            @TASK3_OWNER
/src/server/             @TASK3_OWNER
/src/app/api/            @TASK3_OWNER
```

```

/supabase/ @TASK3_OWNER
/tests/ai/ @TASK3_OWNER
/tests/integration/ @TASK3_OWNER
/docs/workstreams/task-3/ @TASK3_OWNER

# Shared files
/package.json @TASK1_OWNER @TASK2_OWNER @TASK3_OWNER
/pnpm-lock.yaml @TASK1_OWNER @TASK2_OWNER @TASK3_OWNER
/tsconfig.json @TASK1_OWNER @TASK2_OWNER @TASK3_OWNER
/next.config.ts @TASK1_OWNER @TASK2_OWNER @TASK3_OWNER
/vitest.config.ts @TASK1_OWNER @TASK2_OWNER @TASK3_OWNER
/playwright.config.ts @TASK1_OWNER @TASK2_OWNER @TASK3_OWNER
/.env.example @TASK1_OWNER @TASK2_OWNER @TASK3_OWNER
/docs/CONTRACT_SPEC.md @TASK1_OWNER @TASK2_OWNER @TASK3_OWNER
/docs/CONTRACT_CHANGELOG.md @TASK1_OWNER @TASK2_OWNER @TASK3_OWNER

```

6. Ownership Check Script

Create `scripts/check-ownership.mjs`.

It must:

1. Detect the current branch.
2. Compare changed files against `main`.
3. Use an allowlist for each workstream.
4. Fail if files outside the branch's allowed paths were modified.
5. Clearly list unauthorized files.
6. Permit shared files only when an environment variable such as `ALLOW_SHARED_FILES=true` is present.
7. Fail on unknown branches unless explicitly skipped.

Branch allowlists:

```

task-1-product-ui
- src/app/** except src/app/api/**
- src/components/**
- src/features/**
- src/mocks/**
- src/styles/**
- public/**
- tests/e2e/**
- docs/workstreams/task-1/**
- docs/integration-requests/task-1-requests.md

```

```
task-2-finance-engine
- src/core/finance/**
- tests/finance/**
- tests/fixtures/**
- docs/workstreams/task-2/**
- docs/integration-requests/task-2-requests.md

task-3-platform-ai
- src/contracts/**
- src/core/ai/**
- src/server/**
- src/app/api/**
- supabase/**
- tests/ai/**
- tests/integration/**
- docs/workstreams/task-3/**
- docs/integration-requests/task-3-requests.md
- docs/CONTRACT_CHANGELOG.md
```

Task 3 may update `CONTRACT_CHANGELOG.md`, but breaking contract changes still require review.

7. Continuation Documents

Each task needs:

```
STATUS.md
DECISIONS.md
HANDOFF.md
```

STATUS.md template

```
# Task Status

## Current milestone

## Completed

## In progress

## Not started

## Blocked
```

```
## Tests currently passing

## Known failures

## Last verified commit
```

DECISIONS.md template

```
# Technical Decisions

## Decision log

### Decision
- Date:
- Context:
- Decision:
- Alternatives considered:
- Consequences:
- Files affected:
```

HANDOFF.md template

```
# Workstream Handoff

## What currently works

## What was completed most recently

## Exact next task

## Files to read first

## Important implementation details

## Commands to run

## Tests to run

## Known issues

## External requests

## Do not change
```

```
## Last verified commit
```

Every agent must update these files before ending work.

8. Contract Specification

Create `docs/CONTRACT_SPEC.md`.

Document the initial shared conceptual contracts:

```
FinancialProfile
MoneyConstitution
ConstitutionRule
FinancialEvent
DecisionInput
ExtractedDecision
ScenarioInput
ScenarioResult
ScenarioComparison
ConstitutionEvaluation
FutureReceipt
ActionPlan
```

Define the required fields conceptually, including integer-paise money fields.

Use these naming principles:

- Money values end in `Paise`.
- Dates use ISO `YYYY-MM-DD`.
- Date-times use ISO 8601.
- Identifiers are strings.
- Existing fields cannot be renamed or removed without approval.
- New fields should be optional unless jointly approved.
- Every calculated warning uses a stable reason code.
- AI confidence ranges from `0` to `1`.

The Main Setup task should document the contracts but must not implement TypeScript schema files. Task 3 will create the actual Zod schemas.

9. Golden Demo Scenario

Create `docs/DEMO_SCENARIO.md`.

Use one consistent demo user, such as Aarav:

```
Current balance: ₹72,000
Monthly salary: ₹48,000
Salary date: 1st of each month
Rent: ₹14,000
Family transfer: ₹5,000
Existing EMI: ₹3,500
Monthly savings target: ₹8,000
Protected balance floor: ₹10,000
Emergency-fund goal: ₹1,50,000
Proposed phone price: ₹59,999
Down payment: ₹12,000
EMI: ₹4,500
Tenure: 12 months
Processing fee: ₹1,499
```

Document three expected scenarios:

1. Buy now.
2. Wait 45 days.
3. Buy a cheaper ₹39,999 option.

Do not invent final calculated outputs in the Main Setup task. Task 2 will generate verified results.

10. UI Design Principles

Create `docs/UI_DESIGN_PRINCIPLES.md`.

The frontend must feel:

- Sleek
- Modern
- Premium
- Calm
- Trustworthy
- Financially precise
- Spacious
- Responsive

- Production-quality

Visual references may include the product quality and restraint of:

- Linear
- Arc
- ChatGPT
- Stripe
- Apple Wallet
- Premium native finance applications

It must not look like:

- A generic admin dashboard
- A template marketplace dashboard
- A crypto trading screen
- A cluttered banking portal
- A terminal
- A neon AI product
- A hackathon prototype
- A collection of unrelated cards

Design principles:

- The chat is the primary workspace.
- Future Receipt is the signature component.
- The interface should minimize steps.
- Use strong hierarchy and generous spacing.
- Use motion only when it improves comprehension.
- Avoid excessive glassmorphism.
- Avoid excessive gradients.
- Avoid crowded sidebars.
- Avoid fake financial charts.
- Mobile behavior must be intentionally designed.
- Accessibility and keyboard navigation matter.

11. AI Safety Document

Create `docs/AI_SAFETY.md`.

State that AI may:

- Interpret natural language
- Understand Hinglish
- Read screenshots
- Extract purchase details

- Identify missing information
- Suggest candidate scenarios
- Explain verified results
- Draft Constitution rules

AI may not:

- Calculate final balances
- Calculate total EMI commitment
- Calculate goal delay
- Mark a plan safe without finance simulation
- Invent income or commitments
- Activate Constitution rules without approval
- Initiate payments
- Recommend securities
- silently change deterministic outputs

12. Root Configuration

Set up a minimal Next.js TypeScript project using:

- Next.js App Router
- TypeScript
- Tailwind CSS
- ESLint
- Vitest
- Playwright
- Zod

Do not install unnecessary libraries.

Do not add authentication libraries.

Do not add Redux unless implementation later proves it necessary.

Do not add microservices, queues, Kafka, Kubernetes, or a separate Python service.

Add scripts similar to:

```
{
  "dev": "next dev",
  "build": "next build",
  "lint": "next lint",
  "typecheck": "tsc --noEmit",
```

```
"test": "vitest run",
"test:watch": "vitest",
"test:e2e": "playwright test",
"check:ownership": "node scripts/check-ownership.mjs",
"verify:handoff": "node scripts/verify-handoff.mjs",
"smoke": "node scripts/smoke-test.mjs"
}
```

Use the current package manager already selected for the repository. If none exists, use `pnpm`.

13. GitHub Pull Request Template

Include:

```
## Workstream

## What changed

## Owned paths only?

- [ ] Yes

## Shared files changed?

- [ ] No
- [ ] Yes, explained below

## Contract changes

## Tests run

## Screenshots or outputs

## Continuation files updated

- [ ] STATUS.md
- [ ] DECISIONS.md
- [ ] HANDOFF.md

## Integration requests

## Known issues
```

14. Definition of Done

The Main Setup task is complete when:

- Repository structure exists.
- Root dependencies and configuration are valid.
- `pnpm install` works.
- Type checking can run.
- Empty directories are preserved.
- Documentation exists.
- CODEOWNERS exists.
- Pull-request template exists.
- Ownership-check script exists.
- Continuation templates exist.
- Workstream boundaries are documented.
- No product feature implementation has been added.
- Everything is committed to `main`.

Before finishing:

1. Run available checks.
2. Review `git diff --name-only`.
3. Confirm there is no product implementation.
4. Commit with:

```
chore: establish PocketPilot workstream architecture
```

1. Push to `main`.
2. Report exactly what was created and any setup command the three contributors must run.

POCKETPILOT — TASK 1 PROMPT

Product Experience and Frontend

You are responsible for **Task 1: Product Experience and Frontend** for PocketPilot.

Work only on branch:

```
task-1-product-ui
```

Before starting:

1. Pull the latest `main`.
 2. Read:
 3. `README.md`
 4. `AGENTS.md`
 5. `docs/PRODUCT_OVERVIEW.md`
 6. `docs/MVP_SCOPE.md`
 7. `docs/ARCHITECTURE.md`
 8. `docs/CONTRACT_SPEC.md`
 9. `docs/UI_DESIGN_PRINCIPLES.md`
 10. `docs/DEMO_SCENARIO.md`
 11. `docs/workstreams/task-1/STATUS.md`
 12. `docs/workstreams/task-1/DECISIONS.md`
 13. `docs/workstreams/task-1/HANDOFF.md`
 14. Inspect the current code before making changes.
 15. Continue from the last verified state.
 16. Do not restart completed work.
-

Project Overview

PocketPilot is a pre-spend financial decision companion.

A user asks whether they can afford a purchase. PocketPilot collects missing context, runs deterministic financial projections, checks personal financial rules, compares alternative futures, and renders a Future Receipt.

The frontend is not a traditional dashboard.

The primary interaction is a sleek conversational workspace containing rich financial components.

Your Ownership

You may modify only:

```
src/app/** except src/app/api/**
src/components/**
src/features/**
src/mocks/**
src/styles/**
```

```
public/**
tests/e2e/**
docs/workstreams/task-1/**
docs/integration-requests/task-1-requests.md
```

You may read any file.

You may not modify:

```
src/core/finance/**
src/core/ai/**
src/contracts/**
src/server/**
src/app/api/**
supabase/**
tests/finance/**
tests/ai/**
tests/integration/**
```

Do not change shared root configuration without an approved integration request.

Primary Objective

Build the complete PocketPilot MVP product experience using shared contracts when available and high-quality mock data where integration is unfinished.

The UI must work independently before the finance, AI, or backend workstreams are complete.

Use an adapter so mock and live implementations can be switched without rewriting UI components.

Example concept:

```
const pocketPilotClient =
  process.env.NEXT_PUBLIC_USE MOCKS === "true"
    ? mockPocketPilotClient
    : apiPocketPilotClient;
```

Do not call AI providers directly from React components.

Do not perform financial calculations in the frontend.

Do not access a database directly.

Required Product Experience

1. Application shell

Build:

- Calm left navigation
- Main conversational workspace
- Responsive mobile navigation
- Floating multimodal input
- History or saved receipts area
- Constitution access
- Goal visibility
- Demo reset or restart control

Avoid a cluttered dashboard.

2. Onboarding

Create a progressive single-user setup flow for:

- Current balance
- Monthly income
- Salary date
- Rent
- Family transfer
- Existing EMI
- Monthly savings target
- Minimum protected balance
- Emergency-fund goal
- Persona mode

The onboarding should not feel like a long form.

Use progressive disclosure, clear defaults, and a visible sense of progress.

3. Money Constitution experience

Build controls for:

- Minimum balance floor
- Maximum EMI-to-income ratio
- Maximum EMI tenure
- Monthly savings target
- Protected rent
- Protected family transfer

- Guilt-free purchase allowance
- Protected goals

Support:

- Starter rules
- Toggle controls
- Threshold editing
- Draft Rule Card
- Lock, edit, and dismiss actions
- Clear explanation of each rule

4. Decision intake

Support:

- Natural-language text
- Screenshot upload
- Product-link paste
- Manual purchase entry
- Demo purchase

Show upload previews and parsing states.

5. Missing-context interaction

Build:

- Conversational missing-field prompt
- Quick-fill chips
- Custom answer input
- Silent context update
- Automatic continuation of the original decision
- Chips disappearing after selection

6. Safe-to-spend component

Show:

- Safe-to-spend today
- Protected amount
- Upcoming commitment
- Buffer quality
- Confidence
- Last recalculation time

Do not calculate these numbers locally. Render supplied mock or API values.

7. Future Receipt

This is the signature component.

It must display:

- SAFE, CAUTION, WARNING, or BREACH status
- Purchase title
- Listed price
- Upfront payment
- EMI amount and tenure
- Fees
- Actual committed cost
- Commitment ending date
- Lowest projected balance
- Low-balance days
- Negative-balance days
- Goal-delay days
- EMI burden
- Constitution conflicts
- Assumptions
- Confidence
- Recommendation

Include an expandable “See the math” section with:

- Daily ledger preview
- Balance sparkline
- Income markers
- Commitment markers
- Protected-floor line
- Rule-breach markers

8. Parallel Futures

Build an understandable comparison for:

- Buy now
- Wait 45 days
- Buy a cheaper alternative

Each scenario shows:

- Lowest balance
- Risk days
- Goal delay
- Total commitment
- EMI ratio

- Constitution conflicts
- Recommendation

The user must be able to select a preferred future.

9. Safe Purchase Plan

Show:

- Recommended purchase date
- Maximum price
- Required down payment
- Maximum EMI
- Maximum tenure
- Required balance before purchase
- Conditions that invalidate the plan

Support save or approve interaction using mocks until API integration is ready.

10. History states

Build views for:

- Saved Future Receipts
- Approved plans
- Abandoned decisions
- Goal delays avoided
- Constitution breaches prevented

Visual Quality Standard

The interface must feel like a serious modern product, not a hackathon template.

Target qualities:

- Premium
- Sleek
- Calm
- Spacious
- Precise
- Trustworthy
- Fast
- Responsive
- Accessible

Use references for quality, not copying:

- Linear
- Arc
- ChatGPT
- Stripe
- Apple Wallet
- High-quality native finance products

Avoid:

- Generic admin dashboards
- Oversized KPI grids
- Excessive gradients
- Neon purple AI visuals
- Too much glassmorphism
- Tiny text
- Crowded sidebars
- Random cards
- Fake complexity
- Terminal-inspired interfaces
- Excessive animations

Use:

- Strong typography
- Deliberate spacing
- Clear hierarchy
- Subtle borders
- Restrained shadows
- Smooth state transitions
- Meaningful motion
- Excellent empty states
- Excellent loading states
- Excellent error states
- Keyboard accessibility
- Mobile responsiveness

Mock Data

Create complete mocks using the Aarav demo scenario.

The frontend must support a full demo without any backend:

Onboarding

- Constitution
- Ask about phone
- Missing field
- Quick-fill response
- Loading simulation
- Compare futures
- Future Receipt
- Approve safe plan

Keep mock data structurally aligned with `docs/CONTRACT_SPEC.md`.

When Task 3 contracts become available, import them rather than maintaining duplicate public contract definitions.

Cross-Workstream Rules

Before editing a file, verify ownership.

If another workstream must change, do not edit its code.

Record the request in:

```
docs/integration-requests/task-1-requests.md
```

Include:

- Requested change
- Reason
- Required contract
- Files or workstream affected
- Whether it is blocking
- Suggested backward-compatible solution

Do not rename contract fields.

Do not remove contract fields.

Do not change the meaning of contract fields.

Testing

Add or maintain:

- Component interaction tests where appropriate
- Responsive checks
- Keyboard navigation checks
- Critical Playwright flow
- Loading-state tests
- Error-state tests
- Mock-client tests

Run:

```
pnpm typecheck
pnpm lint
pnpm test
pnpm test:e2e
pnpm check:ownership
pnpm build
```

Fix issues inside your ownership area.

Do not bypass checks by modifying another task's code.

Continuation Requirements

Before ending every session, update:

```
docs/workstreams/task-1/STATUS.md
docs/workstreams/task-1/DECISIONS.md
docs/workstreams/task-1/HANDOFF.md
```

The handoff must state:

- What currently works
- What was just completed
- Exact next task
- Files to read first
- Commands to run
- Tests currently passing
- Known visual issues
- Integration requests

- Last verified commit

Another developer or agent must be able to continue without asking what happened previously.

Commit and Push Rules

Use small commits, such as:

```
feat(ui): build PocketPilot application shell
feat(constitution): add rule builder experience
feat(receipt): implement Future Receipt component
feat(scenarios): add parallel futures comparison
test(e2e): cover complete mock decision flow
```

Before committing:

```
git diff --name-only
pnpm check:ownership
```

Confirm every changed file is owned by Task 1.

Push only to:

```
task-1-product-ui
```

Do not merge into `main` directly.

Definition of Done

Task 1 is complete when:

- The entire MVP flow works with mocks.
- The design is sleek and production-quality.
- The experience is not a generic dashboard.
- Future Receipt is polished and understandable.
- Scenario comparison is usable.
- Constitution setup is simple.
- Mobile experience works.
- Loading, empty, and error states exist.
- No financial arithmetic exists in the frontend.
- No direct AI or database calls exist in components.

- E2E demo flow passes.
- Continuation files are current.
- Changes stay inside Task 1 ownership.

Implement the next unfinished milestone, verify it, document it, commit it, and push it to the Task 1 branch.

POCKETPILOT — TASK 2 PROMPT

Deterministic Finance and Money Constitution Engine

You are responsible for **Task 2: Deterministic Finance and Money Constitution Engine**.

Work only on branch:

task-2-finance-engine

Before starting:

1. Pull the latest `main`.
 2. Read:
 3. `README.md`
 4. `AGENTS.md`
 5. `docs/PRODUCT_OVERVIEW.md`
 6. `docs/MVP_SCOPE.md`
 7. `docs/ARCHITECTURE.md`
 8. `docs/CONTRACT_SPEC.md`
 9. `docs/DEMO_SCENARIO.md`
 10. `docs/workstreams/task-2/STATUS.md`
 11. `docs/workstreams/task-2/DECISIONS.md`
 12. `docs/workstreams/task-2/HANDOFF.md`
 13. Inspect existing finance code and tests.
 14. Continue from the last verified state.
 15. Do not restart completed work.
-

Project Overview

PocketPilot shows the future impact of a financial decision before the user spends.

Your subsystem is the trust foundation.

All final monetary calculations must come from deterministic TypeScript code, not from AI.

The same input must always produce the same output.

Your Ownership

You may modify only:

```
src/core/finance/**
tests/finance/**
tests/fixtures/**
docs/workstreams/task-2/**
docs/integration-requests/task-2-requests.md
```

You may read all repository files.

You may not modify:

```
src/app/**
src/components/**
src/features/**
src/mocks/**
src/core/ai/**
src/contracts/**
src/server/**
src/app/api/**
supabase/**
tests/ai/**
tests/integration/**
tests/e2e/**
```

Task 3 owns the implementation of shared contracts.

If a contract problem exists, request a change instead of editing `src/contracts`.

Core Engineering Rules

The finance engine must be:

- Pure TypeScript
- Deterministic

- Independently testable
- Free from React
- Free from AI provider calls
- Free from database dependencies
- Free from server dependencies
- Free from network calls

Store and calculate money using integer paise.

Never use floating-point currency values.

Example:

```
const phonePricePaise = 5_999_900;
```

Use explicit date handling.

Avoid dependence on the machine's local timezone.

Required Module Structure

Implement or continue:

```
src/core/finance/  
├─ index.ts  
├─ types.ts  
├─ errors.ts  
├─ money/  
│   ├─ money.ts  
│   └─ percentage.ts  
├─ dates/  
│   ├─ date-utils.ts  
│   └─ recurrence.ts  
├─ cashflow/  
│   ├─ build-daily-ledger.ts  
│   ├─ project-balance.ts  
│   ├─ calculate-risk-days.ts  
│   └─ calculate-minimum-balance.ts  
├─ emi/  
│   ├─ calculate-emi.ts  
│   ├─ calculate-total-commitment.ts  
│   ├─ calculate-emi-ratio.ts  
│   └─ validate-emi-offer.ts
```

```
├─ constitution/
│   ├── evaluate-rules.ts
│   ├── minimum-balance-rule.ts
│   ├── maximum-emi-ratio-rule.ts
│   ├── maximum-duration-rule.ts
│   ├── savings-target-rule.ts
│   ├── protected-expense-rule.ts
│   └─ goal-protection-rule.ts
├─ goals/
│   ├── calculate-goal-date.ts
│   ├── calculate-goal-delay.ts
│   └─ calculate-required-contribution.ts
├─ scenarios/
│   ├── simulate-buy-now.ts
│   ├── simulate-delay.ts
│   ├── simulate-alternative.ts
│   ├── simulate-income-delay.ts
│   └─ compare-scenarios.ts
├─ safe-to-spend/
│   ├── calculate-safe-to-spend.ts
│   └─ calculate-flexible-income.ts
└─ index.ts
```

Adjust internal organization only inside the owned directory when justified.

Required Capabilities

1. Money primitives

Implement safe operations for:

- Addition
- Subtraction
- Multiplication
- Ratio calculations
- Percentage calculations
- Currency formatting helpers when useful
- Non-negative validation
- Overflow-conscious behavior

2. Daily cash-flow ledger

Generate daily projections supporting:

- Starting balance

- One-time income
- Recurring income
- Salary dates
- One-time expenses
- Recurring expenses
- Rent
- Bills
- Family transfers
- Existing EMIs
- New EMI
- Goal contributions
- Subscriptions
- Purchase down payments
- Processing fees

Each day should expose:

- Date
- Opening balance
- Inflows
- Outflows
- Closing balance
- Applied events
- Risk indicators

3. Simulation horizons

Support:

- 30 days
- 60 days
- 90 days
- 730 days

The API should allow an explicit start and end date or horizon.

4. Safe-to-spend

Calculate:

- Safe-to-spend today
- Flexible income
- Protected commitments
- Protected balance floor
- Amount available until next income
- Buffer remaining
- Confidence-related warnings supplied as input

Do not invent missing financial context.

Return a structured error or incomplete-context result when required data is absent.

5. EMI calculations

Calculate:

- Monthly EMI when inputs permit
- Total instalment commitment
- Processing fees
- Total committed cost
- Commitment ending date
- Existing EMI burden
- New EMI burden
- Total EMI-to-income ratio

Document whether interest rates are monthly or annual and how they are converted.

6. Goal engine

Calculate:

- Original goal completion date
- New completion date
- Goal-delay days
- Required catch-up contribution
- Missed contribution periods
- Goal protection breach

7. Money Constitution evaluation

Support at least:

- Minimum balance floor
- Maximum EMI-to-income ratio
- Maximum EMI tenure
- Monthly savings target
- Priority expense protection
- Protected family transfer
- Goal protection
- Guilt-free purchase allowance
- Income-confidence protection where input is provided

Return structured evaluations, not prose.

8. Warning and reason codes

Use stable reason codes such as:

```
type ScenarioWarningCode =  
  | "MINIMUM_BALANCE_BREACH"  
  | "EMI_RATIO_EXCEEDED"  
  | "EMI_DURATION_EXCEEDED"  
  | "GOAL_DELAYED"  
  | "INCOME_DEPENDENCY"  
  | "NEGATIVE_BALANCE"  
  | "LOW_CONFIDENCE_INCOME"  
  | "PROTECTED_EXPENSE_AT_RISK"  
  | "SAVINGS_TARGET_MISSED"  
  | "RECURRING_COST_UNSUSTAINABLE";
```

Each warning should contain:

- Code
- Severity
- Rule or threshold
- Actual value
- Relevant date or date range
- Human-readable data fields, but not AI-written advice

9. Scenario simulation

Implement:

Buy now

The user's original proposal.

Wait

Simulate waiting 45 days or another requested delay.

Cheaper alternative

Simulate a lower-cost proposal.

Income delay

Stress-test a delayed salary or client payment.

10. Scenario comparison

Compare:

- Minimum balance
- Negative-balance days
- Low-balance days
- Goal delay
- Total commitment
- EMI ratio
- Constitution conflicts
- Protected-expense risk

Return sortable structured results.

Do not produce persuasive language.

Public Finance API

Expose a small stable public API from `src/core/finance/index.ts`.

Target functions:

```
export function simulateScenario(  
  input: ScenarioInput  
): ScenarioResult;  
  
export function compareScenarios(  
  inputs: ScenarioInput[]  
): ScenarioComparison;  
  
export function evaluateConstitution(  
  result: ScenarioResult,  
  constitution: MoneyConstitution  
): ConstitutionEvaluation;  
  
export function calculateSafeToSpend(  
  input: SafeToSpendInput  
): SafeToSpendResult;  
  
export function calculateGoalDelay(  
  input: GoalDelayInput  
): GoalDelayResult;
```

Use the shared contracts when implemented by Task 3.

Until those contracts exist, use internal types aligned exactly with `docs/CONTRACT_SPEC.md` and isolate them so they can be replaced without rewriting calculation logic.

Do not create a competing public contract system.

Golden Fixture

Use the Aarav scenario from `docs/DEMO_SCENARIO.md`.

Create a machine-readable fixture under:

```
tests/fixtures/aarav-phone-purchase.json
```

Generate verified outputs for:

1. Buy now.
2. Wait 45 days.
3. Buy the ₹39,999 alternative.

The fixture and expected outputs become the common truth used by all workstreams.

Do not change demo inputs silently.

Record any correction in `DECISIONS.md`.

Testing Requirements

Add comprehensive tests for:

- Integer-paise precision
- Date boundaries
- Recurring-event scheduling
- Salary timing
- Rent timing
- Processing fees
- EMI total commitment
- EMI ratio
- Minimum balance
- Negative-balance days
- Low-balance days
- Goal delay
- Constitution breaches
- Guilt-free allowance

- Income-delay scenario
- Scenario comparison
- Missing context
- Invalid inputs
- Same-input determinism

Property-based testing may be used where valuable but should not complicate the MVP unnecessarily.

Run:

```
pnpm typecheck
pnpm lint
pnpm test
pnpm check:ownership
pnpm build
```

The finance tests must run without:

- AI credentials
- Database
- Browser
- Network
- Backend server

Cross-Workstream Rules

Before editing, verify file ownership.

If another workstream must change, document it in:

```
docs/integration-requests/task-2-requests.md
```

Do not edit shared contracts directly.

Do not change frontend code to make a test pass.

Do not add server dependencies to finance core.

Do not calculate values in AI prompts.

Do not rename public fields without approval.

Prefer backward-compatible optional additions.

Continuation Requirements

Before ending every session, update:

```
docs/workstreams/task-2/STATUS.md
docs/workstreams/task-2/DECISIONS.md
docs/workstreams/task-2/HANDOFF.md
```

The handoff must include:

- Current public API
- Verified calculations
- Exact next unfinished function
- Files to read
- Commands to run
- Tests passing
- Tests failing
- Known assumptions
- Contract mismatches
- Golden fixture status
- Last verified commit

Another developer or coding agent must be able to continue without reconstructing your reasoning.

Commit and Push Rules

Use focused commits:

```
feat(finance): add daily cash-flow ledger
feat(emi): calculate total committed cost
feat(constitution): evaluate protected balance floor
feat(goals): calculate savings-goal delay
feat(scenarios): compare three purchase futures
test(finance): verify Aarav golden fixture
```

Before committing:

```
git diff --name-only
pnpm check:ownership
```

Push only to:

task-2-finance-engine

Do not push directly to `main`.

Definition of Done

Task 2 is complete when:

- Finance logic is deterministic.
- All money uses integer paise.
- Daily ledgers work.
- Safe-to-spend works.
- EMI calculations work.
- Goal delay works.
- Constitution rules work.
- Three scenarios work.
- Scenario comparison works.
- Stable reason codes exist.
- Golden fixture has verified outputs.
- Unit tests pass independently.
- No AI, React, database, server, or network dependency exists.
- Public API is documented.
- Continuation files are current.
- All changed files stay inside Task 2 ownership.

Implement the next unfinished finance milestone, test it thoroughly, document it, commit it, and push it to the Task 2 branch.

POCKETPILOT — TASK 3 PROMPT

AI, Shared Contracts, Lightweight Backend and Integration

You are responsible for **Task 3: AI, Shared Contracts, Lightweight MVP Backend and Integration**.

Work only on branch:

task-3-platform-ai

Before starting:

1. Pull the latest `main`.

2. Read:

3. `README.md`

4. `AGENTS.md`

5. `docs/PRODUCT_OVERVIEW.md`

6. `docs/MVP_SCOPE.md`

7. `docs/ARCHITECTURE.md`

8. `docs/CONTRACT_SPEC.md`

9. `docs/CONTRACT_CHANGELOG.md`

10. `docs/AI_SAFETY.md`

11. `docs/DEMO_SCENARIO.md`

12. `docs/INTEGRATION_PLAN.md`

13. `docs/workstreams/task-3/STATUS.md`

14. `docs/workstreams/task-3/DECISIONS.md`

15. `docs/workstreams/task-3/HANDOFF.md`

16. Inspect existing contracts, AI, API, persistence, and integration code.

17. Continue from the last verified state.

18. Do not restart completed work.

Project Overview

PocketPilot is a pre-spend financial decision system.

Users submit a proposed purchase using text, screenshots, links, or manual entry.

The system:

1. Extracts a structured decision.
2. Detects missing context.
3. Loads a single demo user's financial profile.
4. Requests deterministic simulations from the finance engine.
5. Evaluates Money Constitution results.
6. Generates a verified Future Receipt explanation.
7. Persists the decision, simulation, receipt, and approved plan.

The MVP does not require authentication.

Explicit MVP Exclusions

Do not implement:

- Login
- Signup

- User accounts
- Sessions
- OAuth
- Passwords
- Multi-tenancy
- Row-Level Security
- Team accounts
- Payment initiation
- Bank account linking
- Production financial aggregation
- Securities recommendations

Assume one seeded local or demo user.

Do not add authentication “for later.” It is intentionally out of scope.

Your Ownership

You may modify only:

```
src/contracts/**
src/core/ai/**
src/server/**
src/app/api/**
supabase/**
tests/ai/**
tests/integration/**
docs/workstreams/task-3/**
docs/integration-requests/task-3-requests.md
docs/CONTRACT_CHANGELOG.md
```

You may read all repository files.

You may not modify:

```
src/components/**
src/features/**
src/mocks/**
src/styles/**
src/core/finance/**
tests/finance/**
tests/e2e/**
```

Do not modify Task 1 UI or Task 2 finance calculations.

Primary Responsibilities

1. Implement shared contracts

Implement Zod and TypeScript contracts for:

- FinancialProfile
- MoneyConstitution
- ConstitutionRule
- FinancialEvent
- DecisionInput
- ExtractedDecision
- ScenarioInput
- ScenarioResult
- ScenarioComparison
- ConstitutionEvaluation
- FutureReceipt
- ActionPlan

Follow `docs/CONTRACT_SPEC.md`.

Contract rules:

- Money uses integer paise.
- Dates use ISO strings.
- Confidence uses `0` to `1`.
- Reason codes are stable strings.
- Existing fields cannot be renamed or removed silently.
- Prefer optional additive changes.
- Export schemas and inferred TypeScript types.
- Reject invalid AI output.

Task 3 owns implementation but not unilateral breaking changes.

Record every contract modification in:

`docs/CONTRACT_CHANGELOG.md`

For each change record:

- Date
- Field or schema
- Change
- Reason
- Breaking or non-breaking

- Affected workstreams
- Migration or adaptation required

2. AI ingestion

Implement a controlled extraction workflow for:

- Natural-language questions
- Hinglish
- Screenshot input
- Product checkout details
- EMI schedules
- Subscription plans
- Manual financial scenarios

Extract:

- Decision type
- Title
- Listed price
- Upfront amount
- Monthly amount
- Tenure
- Processing fee
- Purchase date
- Recurring frequency
- Source evidence
- Confidence
- Missing fields

Validate every model result with Zod.

Do not trust free-form output.

3. Missing-field handling

Implement a deterministic Context Gate.

It should determine whether the system has required:

Profile fields

- Current balance
- Monthly income
- Income timing
- Rent
- Recurring obligations
- Existing EMIs

- Relevant goal
- Relevant Constitution rules

Proposal fields

- Purchase cost
- Upfront cost
- EMI amount when applicable
- Tenure when applicable
- Fees
- Purchase date

When data is missing, return:

- Exact missing field
- User-facing question
- Suggested quick-fill options
- Context update target
- Resume token or request identifier

Ask for one missing dimension at a time.

Do not run finance simulation with invented values.

4. Controlled AI workflow

Use one controlled workflow:

```
Input
→ Extract
→ Validate
→ Check missing fields
→ Generate candidate scenarios
→ Call finance engine
→ Evaluate Constitution
→ Generate verified explanation
→ Persist Future Receipt
```

Do not create an unnecessary swarm of agents.

Use explicit tools or functions for:

- Get financial context
- Extract decision
- Simulate scenario
- Compare scenarios

- Evaluate Constitution
- Save receipt
- Save action plan

5. AI synthesis

Generate persona-adapted explanations for:

- Gen Z Copilot
- Corporate Mentor
- Minimalist

The explanation layer may change tone but must not change numbers.

Every visible calculated claim must map to a field supplied by:

- Finance engine
- Constitution evaluation
- User input
- Extracted source evidence
- Explicit assumption

The synthesis result should be structured for frontend rendering, not merely a long text answer.

6. Lightweight MVP backend

Implement a simple single-user persistence layer.

It may use:

- Supabase/PostgreSQL
- SQLite
- Local JSON for early demo stages

Follow the repository's selected direction.

Do not add auth tables or user-account infrastructure.

Store:

- Demo financial profile
- Money Constitution
- Financial events
- Recurring commitments
- Goals
- Purchase decisions
- Simulation summaries
- Scenario results

- Future Receipts
- Approved Action Plans
- Uploaded artifacts
- AI run logs
- Audit events

Use a fixed demo-user identifier internally when needed.

7. API routes

Implement routes such as:

```
GET  /api/profile
PUT  /api/profile

GET  /api/constitution
PUT  /api/constitution

POST /api/uploads
POST /api/decisions/analyze

POST /api/simulations/create
GET  /api/simulations/[id]

POST /api/plans/approve

POST /api/demo/seed
POST /api/demo/reset
```

Routes must:

- Validate input
- Return shared contract shapes
- Avoid leaking provider errors
- Use consistent error responses
- Avoid duplicating finance calculations
- Call the public finance-core API
- Support mock/demo reliability

8. Upload handling

Support private or local MVP upload handling for:

- PNG
- JPEG
- WebP

Validate:

- MIME type
- File size
- Empty uploads
- Unsupported files

Do not expose secrets or unrestricted file URLs.

9. Demo seeding

Create a one-command or one-endpoint demo seed using the Aarav scenario.

Support:

- Seed
- Reset
- Known phone screenshot
- Known financial profile
- Known Constitution
- Known active goal
- Predictable demo data

Do not hard-code finance results that conflict with Task 2.

When Task 2 output is available, use it as the source of truth.

10. Integration adapters

Provide a stable API client surface for Task 1.

Provide stable finance tool bindings for Task 2.

Example concepts:

```
interface PocketPilotClient {
  getFinancialProfile(): Promise<FinancialProfile>;
  saveMoneyConstitution(
    constitution: MoneyConstitution
  ): Promise<MoneyConstitution>;
  analyzeDecision(
    input: DecisionInput
  ): Promise<DecisionAnalysis>;
  getSimulation(id: string): Promise<ScenarioComparison>;
}
```

```
approvePlan(input: ApprovePlanInput): Promise<ActionPlan>;  
}
```

Do not make Task 1 depend on database details.

Do not make Task 2 depend on server details.

AI Safety Rules

AI may:

- Interpret screenshots
- Understand Hinglish
- Extract terms
- Detect missing fields
- Suggest candidate alternatives
- Explain trade-offs
- Draft a Constitution rule

AI may not:

- Calculate final balances
- Calculate total commitment
- Calculate goal delay
- Declare safety without simulation
- Invent income
- Invent recurring expenses
- Activate a Constitution rule
- Initiate payments
- Recommend securities
- override finance output
- silently hide warnings

Reject unsupported claims.

Expose assumptions and confidence.

Contract Change Protocol

Before changing a public contract:

1. Determine whether the change is breaking.
2. Check Task 1 and Task 2 usage.
3. Prefer an optional field.

4. Add the proposal to `CONTRACT_CHANGELOG.md`.
5. Add a request to:
6. `docs/integration-requests/task-1-requests.md`, or
7. `docs/integration-requests/task-2-requests.md`
8. Do not implement a breaking change until reviewed.

Never rename or remove a field simply because another name looks cleaner.

Testing

Create tests for:

- Contract validation
- Invalid AI outputs
- Screenshot extraction fixtures
- Text extraction
- Hinglish extraction
- Missing-field detection
- Tool selection
- No AI arithmetic
- Explanation fidelity
- Persona routing
- API validation
- Demo seed
- Demo reset
- Persistence
- Finance-core integration
- Error handling
- Upload validation

Run:

```
pnpm typecheck
pnpm lint
pnpm test
pnpm check:ownership
pnpm build
pnpm smoke
```

Tests should use fixtures or mocked AI responses where possible.

Do not require repeated paid model calls in normal CI.

Cross-Workstream Rules

Before editing any file, verify ownership.

If Task 1 needs a UI change, write an integration request.

If Task 2 needs a finance change, write an integration request.

Do not “temporarily” modify another task’s files.

Do not copy finance logic into API handlers.

Do not copy contract definitions into frontend components.

Do not create duplicate public types.

Continuation Requirements

Before ending every session, update:

```
docs/workstreams/task-3/STATUS.md
docs/workstreams/task-3/DECISIONS.md
docs/workstreams/task-3/HANDOFF.md
docs/CONTRACT_CHANGELOG.md
```

The handoff must include:

- Contracts currently implemented
- API routes currently working
- Persistence status
- AI workflow status
- Exact next task
- Files to read
- Commands to run
- Tests passing
- Known failures
- Open contract requests
- Integration status with Tasks 1 and 2
- Last verified commit

Another person or coding agent must be able to continue without asking for previous context.

Commit and Push Rules

Use focused commits:

```
feat(contracts): add core PocketPilot schemas
feat(ai): extract structured purchase decisions
feat(context): add missing-field gate
feat(api): add decision analysis endpoint
feat(demo): seed Aarav financial profile
feat(integration): connect AI workflow to finance core
test(ai): validate extraction fixtures
```

Before committing:

```
git diff --name-only
pnpm check:ownership
```

Push only to:

```
task-3-platform-ai
```

Do not push directly to `main`.

Definition of Done

Task 3 is complete when:

- Shared Zod contracts exist.
- Invalid data is rejected.
- Text and screenshot decisions can be extracted.
- Missing fields are detected.
- AI never performs financial arithmetic.
- Finance-core functions are called for calculations.
- Constitution results are preserved.
- Persona explanations remain numerically faithful.
- Single-user persistence works.
- MVP API routes work.
- Demo seed and reset work.
- No authentication exists.
- Integration tests pass.
- Contract changes are documented.
- Continuation files are current.

- All modified files belong to Task 3.

Implement the next unfinished platform milestone, test it, document it, commit it, and push it to the Task 3 branch.